Конструктор перемещения

```
class Point {
private:
    double x = 0.0;
    double y = 0.0;
public:
    Point() = default;
    Point(double x, double y = 1.4): x(x), y(y) {
        std::cout<< "Constructor" <<std::endl;
    }
    Point(const Point& s): x(s.x), y(s.y) {
        std::cout<< "Copy Constructor" <<std::endl;
    }
    Point(Point &&s): x(s.x), y(s.y) {
        s.x = 0.0;
        s.y = 0.0;
        std::cout<< "Move Constructor" <<std::endl;
    }
};

Point g(Point p) {
    return p;
}

int main () {
   Point p5 = g(Point());
}
```

Конструктор перемещения

Обычно вызывается либо при явном указании std::move:

```
Name p("T");
Name p2 = std::move(p);
```

//p после этого будет невалиден

, либо когда компилятор не применяет оптимизацию и создаёт объект на основании временного:

```
Name func (Name p) {
    Name p1;
    std::cout << "Hi\n";
    return p;
}
int main (){
    Name p("T");
    Name p1 = func(p);
}
```

RVO. Return value optimization

Вот тут, например, будет оптимизация и никакого конструктора перемещения и копирования вызвано не будет (хотя создаётся временный объект и копируется в создающий):

```
Name p1 = Name("Name");
```

Как и тут:

```
Name func () {
    std::cout << "Hi\n";
    return Name("Hello");
}
int main (){
    Name p = func();
}
```

NRVO. Named RVO

Здесь тоже будет выполнена оптимизация:

```
Name func () {
    Name p1("Test");
    std::cout << "Hi\n";
    return p1;
}
int main (){
    Name p = func();
}
```

Здесь адрес переменной p будет неявно передан в функцию и сама переменная p будет связана с возвращаемой переменной из функции p1

Перемещающий оператор присваивания

```
Name& operator= (Name&& o) {
        if (this == &o) return *this;
        delete [] n;
        n = o.n;
        o.n = nullptr;
        return *this;
    }
```

Вызывается либо принудительно std:move

```
int main (){
    Name p = func();
    Name p1;
    p1 = std::move(p);
}
```

Либо при присваивании временного объекта:

```
int main (){
    Name p = func();
    p = Name{"Test"};
}
```

Задачи. Есть ли ошибки? (нет, но конструктор всё же желательно определять)

```
class Point {
    double x = 0;
    double y = 0;
public:
    void setx (double x) {
        this->x = x;
    }
    void sety (double y) {
        this->y = y;
    }
    double getx () const {
        return x;
    }
    double gety () const {
        return y;
    }
};
std::ostream& operator<< (std::ostream& output, const Point& obj) {
    return output << obj.getx() << ' ' << obj.gety() << '\n';
}
int main () {
    Point p;
    p.setx(3.0);
    p.sety(4.6);
    std::cout << p;
}
```

Есть ли ошибки? (да, есть только конструктор с параметрами, а объект создаётся без них)

```
class Point {
    double x = 0;
    double y = 0;
public:
    Point (double x, double y): x(x), y(y) {}
    void setx (double x) {
        this->x = x;
    }
    void sety (double y) {
        this->y = y;
    }
    double getx () const {
        return x;
    }
    double gety () const {
        return y;
    }
};
std::ostream& operator<< (std::ostream& output, const Point& obj) {
    return output << obj.getx() << ' ' << obj.gety() << '\n';
}
int main () {
    Point p;
    p.setx(3.0);
    p.sety(4.6);
    std::cout << p;
}
```

Есть ли ошибки? (да, получилось два конструктора «по умолчанию»)

```
class Point {
    double x = 0;
    double y = 0;
public:
    Point () = default;
    Point (double x = 0, double y = 0): x(x), y(y) {}
    void setx (double x) {
        this->x = x;
    }
    void sety (double y) {
        this->y = y;
    }
    double getx () const {
        return x;
    }
    double gety () const {
        return y;
    }
};
std::ostream& operator<< (std::ostream& output, const Point& obj) {
    return output << obj.getx() << ' ' << obj.gety() << '\n';
}
int main () {
    Point p;
    p.setx(3.0);
    p.sety(4.6);
    std::cout << p;
}
```

```cpp
class Name {
    char* n = new char {};
public:
    Name() { std::cout << "Ctor Empty\n";}
    Name(const char* n): n(new char[strlen(n) + 1]) {
        memcpy (this->n, n, strlen(n));
        std::cout << "Ctor for " << n << '\n' ;
    }
    Name(const Name& other): n(new char[strlen(other.n) + 1]) {
        memcpy (this->n, other.n, strlen(other.n));
        std::cout << "Ctor copy " << n << '\n' ;
    }
    Name& operator=(const Name& other) {
        if (this == &other)
            return *this;
        delete [] n;
        n = new char[strlen(other.n) + 1];
        memcpy (n, other.n, strlen(other.n));
        return *this;
    }
    ~Name() {
       if (n != nullptr) {
           std::cout << "Dtor for " << n << '\n' ;
           delete [] n;
       }
       else  std::cout << "Empty Dtor \n";
    }
    Name (Name&& m) {
        this->n = m.n;
        m.n = nullptr;
        std::cout << "Move Ctor \n" ;
    }
```

```cpp
    Name& operator=(Name&& other) {
            if (this!=&other) {
                n = other.n;
                other.n = nullptr;
            }
            std::cout << "move operator = \n";
            return *this;
} };
```

```cpp
Name func (Name p) {
    Name p1("test_local");
    std::cout << "Hi\n";
    return p;
}
int main () {
    Name test;
    Name test1(test);
    Name test2 = {"test2"};
    test = test2;
    func (test2);
    Name p;
}
```

Что будет выведено на экран?

Результат

```
Ctor Empty
Ctor copy
Ctor for test2
Ctor copy test2
Ctor for test_local
Hi
Move Ctor
Dtor for test_local
Dtor for test2
Empty Dtor
Ctor Empty
Dtor for
Dtor for test2
Dtor for
Dtor for test2
```

При возвращении локальной переменной из функции конструктор копии (и перемещения) не вызывается.

```
Name func (Name p) {
    Name p1("test_local");
    std::cout << "Hi\n";
    return p1;              //поменялось
}
int main () {
    Name test;
    Name test1(test);
    Name test2 = {"test2"};
    test = test2;
    Name test3 = func (test2);//поменялось
    Name p;
}
```

```
Ctor Empty
Ctor copy
Ctor for test2
Ctor copy test2
Ctor for test_local
Hi
Dtor for test2
Ctor Empty
Dtor for
Dtor for test_local
Dtor for test2
Dtor for
Dtor for test2
```

А вот так будет вызван перемещающий оператор =

```
Name func (Name p) {
    Name p1("test_local");
    std::cout << "Hi\n";
    return p1;
}
int main () {
    Name test;
    Name test1(test);
    Name test2 = {"test2"};
    test = test2;
    test1 = func(test2);   //поменялось
    Name p;
}
```

```
Ctor Empty
Ctor copy
Ctor for test2
Ctor copy test2
Ctor for test_local
Hi
move operator =
Empty Dtor
Dtor for test2
Ctor Empty
Dtor for
Dtor for test2
Dtor for test_local
Dtor for test2
```

Перегрузка операторов

Перегрузка функций

Перегрузка - это создание функции с тем же именем, но с отличающимся списком параметров. Иными словами, перегруженные функции *называются одинаково*, но отличаются *количеством или типами своих параметров*. Благодаря этому, компилятор их отличает друг от друга в точке вызова.

Перегрузка операций позволяет разработать **аналог операции для пользовательского типа** в виде функции с определенным набором параметров.

Способы осуществления перегрузки

Перегрузка может быть осуществлена

- в виде **функции-члена** класса (методом)
- в виде **глобальной функции** (обычной или дружественной классу)

Пример использования перегрузки при сложении матриц:

```
Matrix a, b, c;
...
c = a + b;
```

Правила перегрузки

Для успешного использования перегрузки, надо знать и соблюдать ряд правил:

- Перегружать можно все имеющиеся операции, кроме
  ?: - тернарный оператор;
  :: - доступ к вложенным именам;
  . - доступ к полям;
  .* - доступ к полям по указателю;
  sizeof, typeid и операторы cast.

Подробнее здесь.

Правила перегрузки (продолжение)

- **Нельзя придумывать** свои операции, например,
  @, +++,)(, (_!_) и др.
- при перегрузке **нельзя изменять** арность операции, ассоциативность, приоритет
- способы перегрузки унарных и бинарных операций **отличаются**
- хотя бы один из аргументов перегружаемых оператором должен **быть пользовательского типа**

Правила перегрузки (продолжение)

- следующие операторы можно перегрузить **только в качестве методов**:
  = - присваивание;
  -> - доступ к полям по указателю;
  () - вызов функции;
  [] - доступ по индексу;
  ->* - доступ к указателю-на-поле по указателю;
  операторы конверсии и управления памятью.
- следующие операторы можно перегрузить **только в виде внешних функций** (перегружаем методы класса <iostream>):
  чтение из потока;
  запись в поток.

Перегрузки методом класса: бинарные операции

Для перегрузки бинарных операций нужно создать метод, принимающий один аргумент – **ссылку на второй аргумент операции**.

```
class Complex {
private:
  double _re = 0, _im = 0;
public:
    Complex() = default;
    Complex(double re, double im) : _re(re), _im(im) {}
    Complex operator+ (const Complex& c) {
        return Complex(_re + c._re, _im + c._im);
    }
};
```

Пример вызова

```
Complex a(1, 2), b;
b + a;
b.operator+(a);
```

В результате операции сложения возникает новый объект типа Complex

Оператор присваивания

Операция **присваивания** определена в любом классе по умолчанию как поэлементное копирование. Если класс содержит поля, память под которые выделяется **динамически**, необходимо определить собственную операцию присваивания. Чтобы сохранить семантику присваивания, функция должна **возвращать ссылку на объект**, для которого она вызвана и **принимать** в качестве параметра **ссылку на присваиваемый объект**.

- Оператор = можно перегружать **только методом класса**.
- Оператор = при наследовании **не наследуется**. Если подумать, то будет видно, что при наследовании могут быть добавлены новые поля или изменено их смысловое значение и следовательно оператор = может не работать корректно.

Бинарные операции (продолжение)

Перегрузим присваивание:

```
class Complex {
        . . .

    Complex& operator=(const Complex& c) {
        _re = c._re;
        _im = c._im;
        return *this;
    }
        . . .
};
```

Теперь можно воспользоваться двумя перегруженными операциями:

```
Complex x(3,7), y(2,7), z;
z = x + y;
```

Результат выполнения программы?

```
class Complex {
private:
    int _re, _im;
public:
    Complex(int re = 0, int im = 0) : _re(re), _im(im) {
        std::cout << "C(" << re << "," << im << ")" << std::endl;
    }
    ~Complex() {
        std::cout << "D(" << _re << "," << _im << ")" << std::endl;
    }
    Complex& operator=(const Complex& c) {
        _re = c._re; _im = c._im;
        return *this;
    }
    Complex operator+(const Complex& c) {
        Complex temp(_re + c._re, _im + c._im);
        return temp;
    }
};


int main () {
    Complex a(1, 2), b;
    b = a + a;
}
```

Результат:
C(1,2)
C(0,0)
C(2,4)
D(2,4)
D(2,4)
D(1,2)

А сейчас что будет?

```
class Complex {
private:
    int _re, _im;
public:
    Complex(int re = 0, int im = 0) : _re(re), _im(im) {
        std::cout << "C(" << re << "," << im << ")" << std::endl;
    }
    ~Complex() {
        std::cout << "D(" << _re << "," << _im << ")" << std::endl;
    }
    Complex& operator=(const Complex& c) {
        _re = c._re; _im = c._im;
        return *this;
    }
    Complex operator+(const Complex& c) {
        Complex temp(_re + c._re, _im + c._im);
        return temp;
    }
};


int main () {
    Complex a(1, 2), b;
    a + b = b;
}
```

Результат:

C(1,2)
C(0,0)
C(1,2)
D(1,2)
D(0,0)
D(1,2)

Немного про rvalue и lvalue

- *lvalue (locator value)* представляет собой объект, который занимает идентифицируемое место в памяти (например, имеет адрес).
- *rvalue* определено путём исключения, говоря, что любое выражение является либо *lvalue*, либо *rvalue*. Таким образом из определения *lvalue* следует, что *rvalue* — это выражение, которое *не обязательно представляет* собой объект, у которого можно определить адрес. Грубо говоря, rvalue может быть временным объектом, а lvalue - нет.

```
int& foo () {
    return 2;
}

int main () {
    int var;
    var = 4;

    6 = var;
    (var + 1) = 4;

}
```

Non-const lvalue reference to type 'int' cannot bind to a temporary of type 'int'

Expression is not assignable

Expression is not assignable

Методы применимы для любых видов value

Но можно указать, что оператор присваивания применим только к lvalue

```
Complex& operator= (const Complex& c) & {
        _re = c._re; _im = c._im;
        return *this;
    }
```

Перегрузка методом класса: унарные операции

Поскольку в операции участвует только один операнд, то никаких внешних ссылок методу, реализующему операцию, передавать не нужно.

```
Complex operator-() {
      Complex temp;
      temp._re = -_re;
      temp._im = -_im;
      return temp;
    }
```

Для операций ++ и -- также существуют две формы: **префиксная** и **постфиксная**, которые реализуются при перегрузке особым образом.

Пример перегрузки унарных операций

```
class Coord {
  int _x, _y, _z;


 public:
  Coord(int x = 0, int y = 0, int z = 0) : _x(x), _y(y), _z(z){}
  ...
  // Перегрузка префиксной формы:
  Coord& operator++() {
    ++_x; ++_y; ++_z;
    return *this;
  }
  // Перегрузка постфиксной формы:
  Coord operator++(int) {
    Coord temp = *this;
    ++_x; ++_y; ++_z;
    return temp;
  }
  ...
};
```

Перегрузка внешней функцией

В этом случае у нас есть также две возможности: создать обычную функцию, или функцию, **дружественную** классу. В случае **унарной** операции функция должна принимать ссылку на объект класса, к которому она применяется. Если операция **бинарная**, то таких ссылок должно быть две: на первый аргумент и на второй.

Пример дружественной функции

```
class Coord {
  int _x = 0, _y = 0, _z = 0;

public:
  Coord() {}
  Coord(int x, int y, int z) : _x(x), _y(y), _z(z) {}
  friend Coord operator+(const Coord&, const Coord&);
};


Coord operator+(const Coord& c1, const Coord& c2) {
  Coord temp;
  temp._x = c1._x + c2._x;
  temp._y = c1._y + c2._y;
  temp._z = c1._z + c2._z;
  return temp;
}

int main() {
  Coord a(1, 2, 3), b(4, 5, 6), c;
  c = a + b;
}
```

Перегрузка потоков ввода/вывода

- Функции operator<< и operator>> могут быть **друзьями класса**, описывающего **пользовательский тип**.
- Функции **принимают** в качестве аргументов **ссылку на входной (выходной) поток** и **ссылку на объект класса**, данные из которого необходимо обработать. В качестве **возвращаемых** значений необходимо также указывать **ссылки на поток** ввода или вывода (в этом случае можно конструировать сложные потоковые конструкции).

Пример

```
class Complex {
  ...
  friend ostream& operator<<(ostream& os, Complex& c);
  friend istream& operator>>(istream& is, Complex& c);
  ...
};
ostream& operator<<(ostream& os, const Complex& c) {
  return os << '(' << c._re << ',' << c._im << ')';
}
istream& operator>>(istream& is, Complex& c) {
       return is >> c._re >> c._im;
}
int main() {
  Complex c(0, 0);
  std::cin >> c;  // 23,45
  std::cout << c; // (23,45)
}
```

Перегрузка операции преобразования типов

Преобразования типов позволяют задать правила преобразования нашего класса к другим типам и классам.

Например:

```
Point::operator bool() const {
  return this->x != 0 || this->y != 0;
}
```

```
class Name {
    char* n = new char {};
    size_t size = 0;
public:
    Name() {}
    Name(const char* n): n(new char[strlen(n) + 1]), size(strlen(n)) {
        memcpy (this->n, n, strlen(n));
    }
    Name(const Name& other): n(new char[strlen(other.n) + 1]), size(strlen(other.n))  {
        memcpy (this->n, other.n, strlen(other.n));
    }
    Name& operator=(const Name& other) = delete;
    ~Name() {
       if (n != nullptr) {
           delete [] n;
       }
    }
    operator char* () const {
        return n;
        }
    operator size_t () const {
        return size;
    }};
int main () {
    Name name1("Hello");
    char string[20];
    std::strcpy (string, name1);
    std::strcpy (name1, "Bye");
    std::cout << static_cast <char *> (name1) << '\n’;   //Bye
    size_t size = 30;
    std::cout << name1 + size << '\n’; }           //35
```

Ещё про преобразование типов

Если в классе определён конструктор, принимающий на вход определённый тип данных, то переменная этого типа может быть неявно преобразована к классу (будет неявно вызван конструктор этого класса).

Таким образом, имея конструктор, принимающий, например, const char*

```
Name(const char* n): n(new char[strlen(n) + 1]), size(strlen(n)) {
        memcpy (this->n, n, strlen(n));
    }
```

Теперь переменная этого типа может быть преобразована к типу Name.

```
class Name {
    char* n = new char {};
public:
    Name() { std::cout << "Ctor Empty\n";}
    Name(const char* n): n(new char[strlen(n) + 1]) {
        memcpy (this->n, n, strlen(n));
    }
    Name(const Name& other): n(new char[strlen(other.n) + 1]) {
        memcpy (this->n, other.n, strlen(other.n));
    }
    Name& operator=(const Name& other) = delete;
    ~Name() {
       if (n != nullptr) {
           delete [] n;
       }
    }
    operator char* () const {
        return n;
    }
    Name operator+ (const Name& right) {
        Name left_copy = *this;
        return strcat(left_copy.n, right.n);
    }
};
int main () {
    Name name1("Hello");
    char string[20]{" world!"};
    std::cout << name1 + string;      //Порядок операций?
};
```

Порядок операций с предыдущего слайда

1. Преобразование переменной string в тип Name, то есть создание нового объекта на основании переданной строки

2. Вызов оператора +

3. В операторе будет вызван конструктор копирования для создания локальной переменной left_copy

4. Для возвращения объекта Name из оператора + будет вызван конструктор для преобразования результата strcat в тип Name

5. На выходе будет вызван деструктор локальной переменной (left_copy)

6. Для вставки в поток результат работы оператора + будет преобразован в строку (char*) с помощью оператора operator char *

7. Вывод на экран

8. Будет вызван деструктор для строки, которая была создана в результате strcat

9. Будет вызван деструктор для объекта, в который была преобразована переменная string

И опять про перегрузку операторов

```
class Name {
    char* n = new char {};
public:
        . . .
    friend std::ostream& operator<< (std::ostream& out, const Name& obj);
    friend Name operator+ (const Name& left, const Name& right);
};

std::ostream& operator<< (std::ostream& out, const Name& obj){
    out << obj.n << '\n';
    return out;
}
Name operator+ (const Name& left, const Name& right) {
    Name left_copy = left;
    return strcat(left_copy.n, right.n);
}
int main () {
    Name name("Hello");
    char string[20]{" world!"};
    std::cout <<  string + name;
    std::cout <<  name + string;
};
```

Перегрузка операторов + и += (не оптимальный)

```
class Point {
public:
    double x = 5.0;
    double y = 6.0;
    Point() = default;
    Point(double x, double y = 1.4): x(x), y(y) {}
    Point& operator += (const Point& right) {
        *this = *this + right;
        return *this;
    }
    friend Point operator+ (const Point& left, const Point& right);
};


Point operator+ (const Point& left, const Point& right) {
    return Point (left.x + right.x, left.y + right.y);
}


int main () {
    Point p(1,6);
    Point p1(2,7);
    p+=p1;
    std::cout << p.x << ' ' << p.y << '\n';
};
```

Лучше + выражать через +=

```
class Point {
public:
    double x = 5.0;
    double y = 6.0;
    Point() = default;
    Point(double x, double y = 1.4): x(x), y(y) {}
    Point& operator += (const Point& right) {
        x+=right.x;
        y+=right.y;
        return *this;
    }
    friend Point operator+ (const Point& left, const Point& right);
};
Point operator+ (const Point& left, const Point& right) {
    Point temp = left;
    temp+=right;
    return temp;
}
int main () {
    Point p(1,6);
    Point p1(2,7);
    p+=p1;
    std::cout << p.x << ' ' << p.y << '\n';
};
```

А оператор > выражать через оператор < ☺

```
bool operator < (const Point& left, const Point& right) {
    return (left.x < right.x) && (left.y < right.y);
}


bool operator > (const Point& left, const Point& right) {
    return right < left;
}
```